

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Programas Clase 22

Unidad de Aprendizaje: *Sistemas Distribuidos*

Profesor: *Coronilla Contreras Ukranio*

Alumnos:

De la Rosa Gutiérrez Kaleb Yael

Grupo*: 7CM2*

Ejercicio 1

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/*
 Clase Application para el Ejercicio 1 (Cliente HTTP Asíncrono Despachador).
 */
public class Application {
    private static final String WORKER_ADDRESS_1 =
"http://localhost:8081/task";
    private static final String WORKER_ADDRESS_2 =
"http://localhost:8082/task";

    public static void main(String[] args) {
        String hashObjetivo =
"de3925d6e4321394d09e782b1c517c4ead2ea4d1209439ece3c42ac43db6d31f";
        if (args.length > 0) {
            hashObjetivo = args[0].trim();
        }

        Aggregator aggregator = new Aggregator();

        // Generar las 26 tareas correspondientes al abecedario de 'a' a 'z'
        List<String> listaTareas = new ArrayList<String>();
        for (char c = 'a'; c <= 'z'; c++) {
            listaTareas.add(hashObjetivo + "," + c);
        }

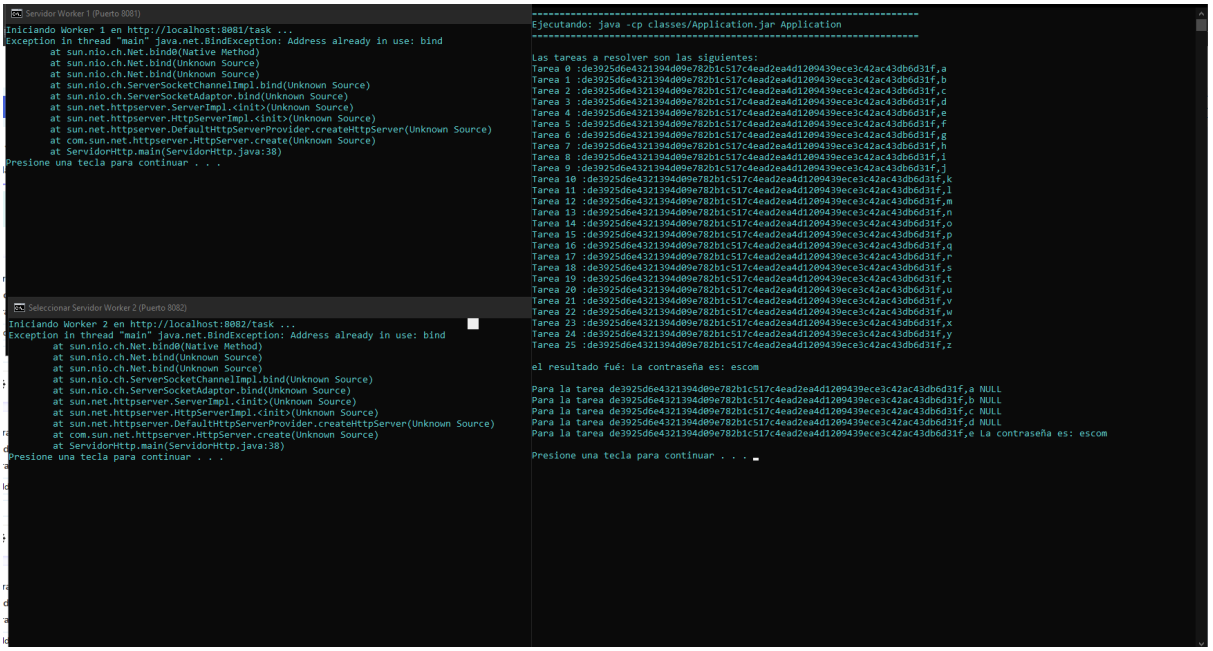
        // Impresión inicial de las tareas a resolver
        System.out.println("Las tareas a resolver son las siguientes:");
        for (int i = 0; i < listaTareas.size(); i++) {
            System.out.println("Tarea " + i + " :" + listaTareas.get(i));
        }
        System.out.println();

        // Envío distribuido de tareas hacia los dos servidores trabajadores
        List<String> results = aggregator.sendTasksToWorkers(
                Arrays.asList(WORKER_ADDRESS_1, WORKER_ADDRESS_2),
                listaTareas);

        // Identificar la solución encontrada entre los resultados devueltos
```

```java
        String resultadoFinal = "NULL";
        for (String r : results) {
            if (r != null && !r.trim().equalsIgnoreCase("NULL") &&
!r.trim().isEmpty()) {
                resultadoFinal = r.trim();
                break;
            }
        }

        // Impresión del resultado final y del detalle de tareas realizadas
        System.out.println("el resultado fué: " + resultadoFinal);
        System.out.println();
        for (int i = 0; i < results.size(); i++) {
            System.out.println("Para la tarea " + listaTareas.get(i) + " " +
results.get(i).trim());
        }
    }
}
```



Ejercicio 2

```
import com.sun.net.httpserver.Headers;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.concurrent.Executors;

/**
Ejercicio 2: Servidor Web HTTP Despachador (WebServer).
 */
public class WebServer {

    private static final int PUERTO_DEFAULT = 8080;
    private static final int NUM_HILOS = 8;
    private static final String WORKER_ADDRESS_1 =
"http://localhost:8081/task";
    private static final String WORKER_ADDRESS_2 =
"http://localhost:8082/task";

    public static void main(String[] args) throws IOException {
        int puerto = PUERTO_DEFAULT;
        if (args.length > 0) {
            try {
                puerto = Integer.parseInt(args[0]);
            } catch (NumberFormatException e) {
                System.out.println("Puerto no válido. Usando puerto por
defecto: " + PUERTO_DEFAULT);
                puerto = PUERTO_DEFAULT;
            }
        }
```

```
        HttpServer server = HttpServer.create(new InetSocketAddress(puerto),
0);
        server.setExecutor(Executors.newFixedThreadPool(NUM_HILOS));

        server.createContext("/dispatcher", new DispatcherHandler());
        server.createContext("/", new StatusHandler());

        server.start();

System.out.println("============================================================
=======");
        System.out.println(" Servidor Despachador (WebServer) iniciado en el
puerto " + puerto);
        System.out.println(" Endpoint disponible: http://localhost:" + puerto
+ "/dispatcher");
        System.out.println(" Workers configurados:");
        System.out.println("   - Worker 1: " + WORKER_ADDRESS_1);
        System.out.println("   - Worker 2: " + WORKER_ADDRESS_2);
        System.out.println(" Esperando solicitudes de clientes...");

System.out.println("============================================================
=======");
    }

    static class DispatcherHandler implements HttpHandler {

        @Override
        public void handle(HttpExchange exchange) throws IOException {
            long nanoInicio = System.nanoTime();

            String metodo = exchange.getRequestMethod();
            Headers headers = exchange.getRequestHeaders();

            // Detectar cabecera X-Debug: true
            String xDebugValor = headers.getFirst("X-Debug");
            boolean esDebug = "true".equalsIgnoreCase(xDebugValor);

            // Soporte para CORS (útil para integración con Frontend Web en
Ejercicio 3)
            Headers resHeaders = exchange.getResponseHeaders();
            resHeaders.set("Access-Control-Allow-Origin", "*");
```

```
            resHeaders.set("Access-Control-Allow-Methods", "POST, GET,
OPTIONS");
            resHeaders.set("Access-Control-Allow-Headers", "Content-Type,
X-Debug");

            if ("OPTIONS".equalsIgnoreCase(metodo)) {
                exchange.sendResponseHeaders(204, -1);
                return;
            }

            if ("POST".equalsIgnoreCase(metodo)) {
                // Leer el cuerpo de la petición
                InputStream is = exchange.getRequestBody();
                ByteArrayOutputStream buffer = new ByteArrayOutputStream();
                byte[] temp = new byte[1024];
                int leidos;
                while ((leidos = is.read(temp, 0, temp.length)) != -1) {
                    buffer.write(temp, 0, leidos);
                }
                String cuerpo = new String(buffer.toByteArray(),
StandardCharsets.UTF_8).trim();

                System.out.println("\n[POST /dispatcher] Solicitud recibida:
<" + cuerpo + ">");

                // Extraer el hashObjetivo (soporta tanto 'hash' como
'hash,letra')
                String hashObjetivo;
                if (cuerpo.contains(",")) {
                    hashObjetivo = cuerpo.split(",")[0].trim();
                } else {
                    hashObjetivo = cuerpo.trim();
                }

                if (hashObjetivo.isEmpty()) {
                    String error = "Error: Cuerpo de petición vacío.\n";
                    byte[] errorBytes =
error.getBytes(StandardCharsets.UTF_8);
                    exchange.sendResponseHeaders(400, errorBytes.length);
                    OutputStream os = exchange.getResponseBody();
                    os.write(errorBytes);
                    os.close();
```

```
                    return;
                }

                System.out.println(">> Iniciando despacho de 26 tareas para
hash: " + hashObjetivo);

                // Instanciar Aggregator y generar la lista de 26 tareas
                Aggregator aggregator = new Aggregator();
                List<String> listaTareas = new ArrayList<>();
                for (char c = 'a'; c <= 'z'; c++) {
                    listaTareas.add(hashObjetivo + "," + c);
                }

                // Enviar las tareas distribuidas hacia los dos workers
                List<String> results = aggregator.sendTasksToWorkers(
                        Arrays.asList(WORKER_ADDRESS_1, WORKER_ADDRESS_2),
                        listaTareas);

                // Encontrar la solución entre los resultados obtenidos
                String respuestaCliente = "NULL";
                for (String r : results) {
                    if (r != null && !r.trim().equalsIgnoreCase("NULL") &&
!r.trim().isEmpty()) {
                        respuestaCliente = r.trim();
                        break;
                    }
                }

                // Asegurar formato estándar de respuesta con salto de línea
final
                if (!respuestaCliente.endsWith("\n")) {
                    respuestaCliente += "\n";
                }

                System.out.println(">> Resultado obtenido: " +
respuestaCliente.trim());

                // Si se solicitó X-Debug, calcular duración y adjuntar
cabecera
                if (esDebug) {
                    long nanoFin = System.nanoTime();
                    long duracionNanos = nanoFin - nanoInicio;
```

```
                    long segundos = duracionNanos / 1_000_000_000L;
                    long milisegundos = (duracionNanos % 1_000_000_000L) /
1_000_000L;

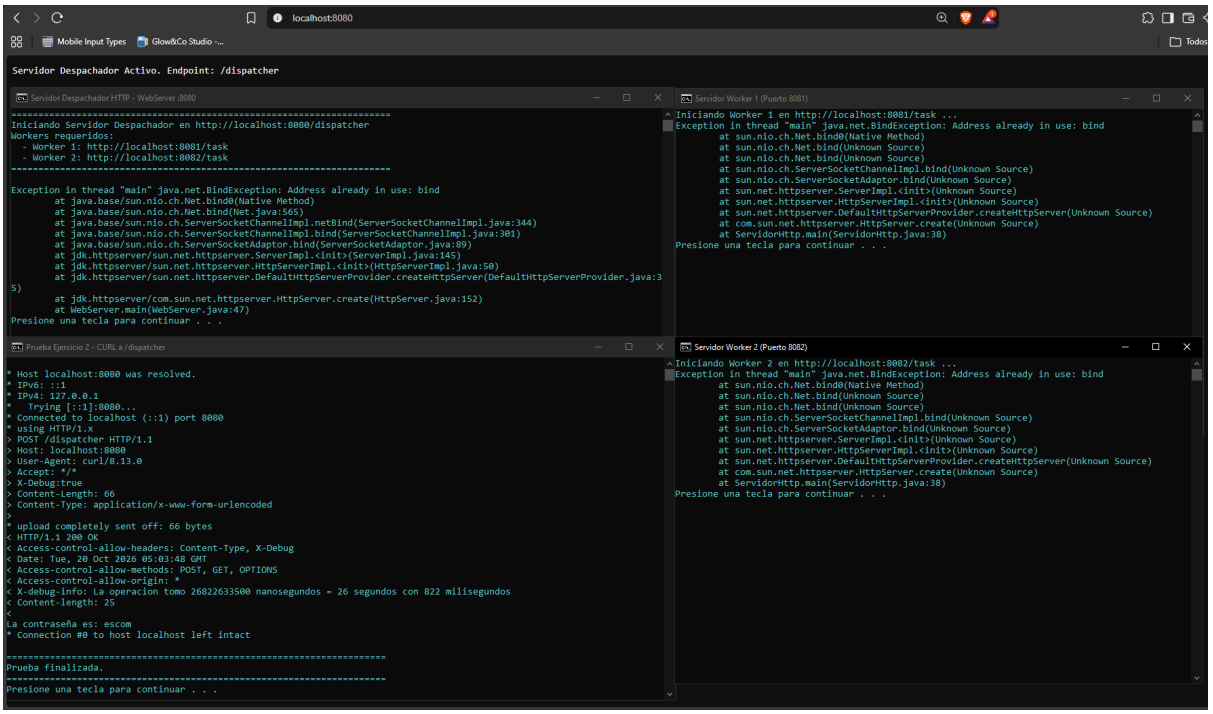
                    String debugInfo = String.format(
                            "La operacion tomo %d nanosegundos = %d segundos
con %d milisegundos",
                            duracionNanos, segundos, milisegundos
                    );
                    resHeaders.set("X-debug-info", debugInfo);
                    System.out.println(">> " + debugInfo);
                }

                byte[] bytesRespuesta =
respuestaCliente.getBytes(StandardCharsets.UTF_8);
                exchange.sendResponseHeaders(200, bytesRespuesta.length);
                OutputStream os = exchange.getResponseBody();
                os.write(bytesRespuesta);
                os.close();
                return;
            }

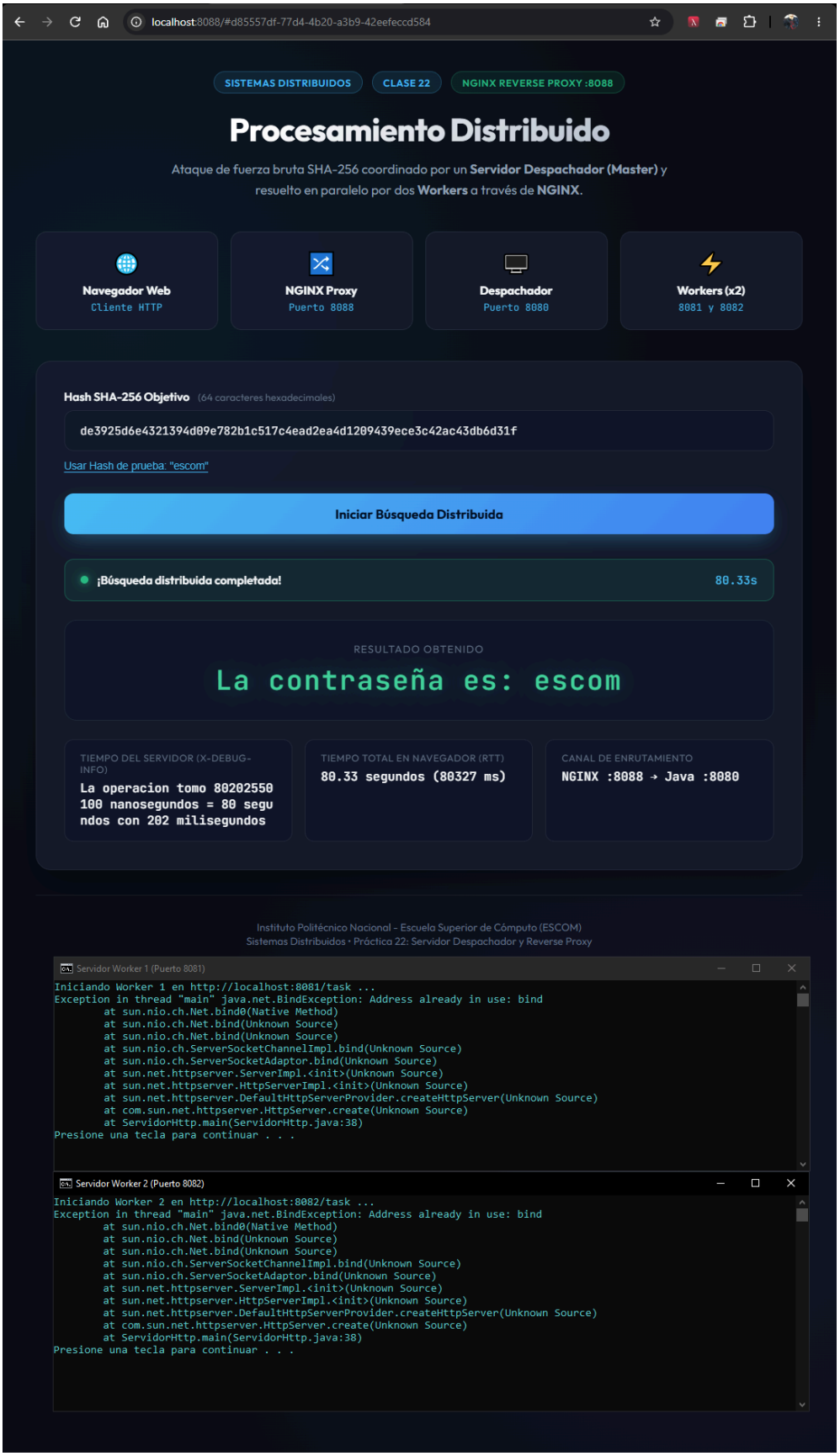
            // Para otros métodos (ej. GET)
            String msg = "Servidor Despachador HTTP activo. Envíe una
solicitud POST a /dispatcher con el hashObjetivo.\n";
            byte[] msgBytes = msg.getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(200, msgBytes.length);
            OutputStream os = exchange.getResponseBody();
            os.write(msgBytes);
            os.close();
        }
    }

    static class StatusHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String msg = "Servidor Despachador Activo. Endpoint:
/dispatcher\n";
            byte[] msgBytes = msg.getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(200, msgBytes.length);
            OutputStream os = exchange.getResponseBody();
            os.write(msgBytes);
```

```
                os.close();
            }
        }
}
```


localhost:8080
Mobile Input Types
Glow&Co Studio -...
Todos
Servidor Despachador Activo. Endpoint: /dispatcher
Servidor Despachador HTTP - WebServer :8080
Iniciando Servidor Despachador en http://localhost:8080/dispatcher
Workers requeridos:
- Worker 1: http://localhost:8081/task
- Worker 2: http://localhost:8082/task
Exception in thread "main" java.net.BindException: Address already in use: bind
at java.base/sun.nio.ch.Net.bind0(Native Method)
at java.base/sun.nio.ch.Net.bind(Net.java:565)
at java.base/sun.nio.ch.ServerSocketChannelImpl.netBind(ServerSocketChannelImpl.java:344)
at java.base/sun.nio.ch.ServerSocketChannelImpl.bind(ServerSocketChannelImpl.java:301)
at java.base/sun.nio.ch.ServerSocketAdaptor.bind(ServerSocketAdaptor.java:89)
at jdk.httpserver/sun.net.httpserver.ServerImpl.<init>(ServerImpl.java:145)
at jdk.httpserver/sun.net.httpserver.HttpServerImpl.<init>(HttpServerImpl.java:50)
at jdk.httpserver/sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(DefaultHttpServerProvider.java:35)
at jdk.httpserver/com.sun.net.httpserver.HttpServer.create(HttpServer.java:152)
at WebServer.main(WebServer.java:47)
Presione una tecla para continuar . . .
Servidor Worker 1 (Puerto 8081)
Iniciando Worker 1 en http://localhost:8081/task ...
Exception in thread "main" java.net.BindException: Address already in use: bind
at sun.nio.ch.Net.bind0(Native Method)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.ServerSocketChannelImpl.bind(Unknown Source)
at sun.nio.ch.ServerSocketAdaptor.bind(Unknown Source)
at sun.net.httpserver.ServerImpl.<init>(Unknown Source)
at sun.net.httpserver.HttpServerImpl.<init>(Unknown Source)
at sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(Unknown Source)
at com.sun.net.httpserver.HttpServer.create(Unknown Source)
at ServidorHttp.main(ServidorHttp.java:38)
Presione una tecla para continuar . . .
Prueba Ejercicio 2 - CURL a /dispatcher
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* using HTTP/1.x
> POST /dispatcher HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.13.0
> Accept: */*
> X-Debug:true
> Content-Length: 66
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 66 bytes
< HTTP/1.1 200 OK
< Access-control-allow-headers: Content-Type, X-Debug
< Date: Tue, 20 Oct 2026 05:03:48 GMT
< Access-control-allow-methods: POST, GET, OPTIONS
< Access-control-allow-origin: *
< X-debug-info: La operacion tomo 26822633500 nanosegundos = 26 segundos con 822 milisegundos
< Content-length: 25
<
La contraseña es: escom
* Connection #0 to host localhost left intact
Prueba finalizada.
Presione una tecla para continuar . . .
Servidor Worker 2 (Puerto 8082)
Iniciando Worker 2 en http://localhost:8082/task ...
Exception in thread "main" java.net.BindException: Address already in use: bind
at sun.nio.ch.Net.bind0(Native Method)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.ServerSocketChannelImpl.bind(Unknown Source)
at sun.nio.ch.ServerSocketAdaptor.bind(Unknown Source)
at sun.net.httpserver.ServerImpl.<init>(Unknown Source)
at sun.net.httpserver.HttpServerImpl.<init>(Unknown Source)
at sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(Unknown Source)
at com.sun.net.httpserver.HttpServer.create(Unknown Source)
at ServidorHttp.main(ServidorHttp.java:38)
Presione una tecla para continuar . . .

Ejercicio 3


localhost:8088/#d85557df-77d4-4b20-a3b9-42eefeccd584
SISTEMAS DISTRIBUIDOS
CLASE 22
NGINX REVERSE PROXY :8088
Procesamiento Distribuido
Ataque de fuerza bruta SHA-256 coordinado por un Servidor Despachador (Master) y resuelto en paralelo por dos Workers a través de NGINX.
Navegador Web
Cliente HTTP
NGINX Proxy
Puerto 8088
Despachador
Puerto 8080
Workers (x2)
8081 y 8082
Hash SHA-256 Objetivo (64 caracteres hexadecimales)
de3925d6e4321394d09e782b1c517c4ead2ea4d1209439ece3c42ac43db6d31f
Usar Hash de prueba: "escom"
Iniciar Búsqueda Distribuida
¡Búsqueda distribuida completada!
80.33s
RESULTADO OBTENIDO
La contraseña es: escom
TIEMPO DEL SERVIDOR (X-DEBUG-INFO)
La operacion tomo 80202550 100 nanosegundos = 80 segundos con 202 milisegundos
TIEMPO TOTAL EN NAVEGADOR (RTT)
80.33 segundos (80327 ms)
CANAL DE ENRUTAMIENTO
NGINX :8088 → Java :8080
Instituto Politécnico Nacional - Escuela Superior de Cómputo (ESCOM)
Sistemas Distribuidos · Práctica 22: Servidor Despachador y Reverse Proxy
Servidor Worker 1 (Puerto 8081)
Iniciando Worker 1 en http://localhost:8081/task ...
Exception in thread "main" java.net.BindException: Address already in use: bind
at sun.nio.ch.Net.bind0(Native Method)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.ServerSocketChannelImpl.bind(Unknown Source)
at sun.nio.ch.ServerSocketAdaptor.bind(Unknown Source)
at sun.net.httpserver.ServerImpl.<init>(Unknown Source)
at sun.net.httpserver.HttpServerImpl.<init>(Unknown Source)
at sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(Unknown Source)
at com.sun.net.httpserver.HttpServer.create(Unknown Source)
at ServidorHttp.main(ServidorHttp.java:38)
Presione una tecla para continuar . . .
Servidor Worker 2 (Puerto 8082)
Iniciando Worker 2 en http://localhost:8082/task ...
Exception in thread "main" java.net.BindException: Address already in use: bind
at sun.nio.ch.Net.bind0(Native Method)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.Net.bind(Unknown Source)
at sun.nio.ch.ServerSocketChannelImpl.bind(Unknown Source)
at sun.nio.ch.ServerSocketAdaptor.bind(Unknown Source)
at sun.net.httpserver.ServerImpl.<init>(Unknown Source)
at sun.net.httpserver.HttpServerImpl.<init>(Unknown Source)
at sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(Unknown Source)
at com.sun.net.httpserver.HttpServer.create(Unknown Source)
at ServidorHttp.main(ServidorHttp.java:38)
Presione una tecla para continuar . . .